

Compiler Construction

⋈ Backend ⋈

Memory Hierarchy

Different kinds of memory in a computer, with different performance:

Registers Small memory units built on the CPU (bytes, 1 cycle)

L1 Cache Last main memory access results (kB, 2-3 cycles)

L2 Cache (MB, 10 cycles)

Memory The usual RAM (GB, 100 cycles)

Storage Disks (100GB, TB, > 1Mcycles)

Memory Hierarchy

Different kinds of memory in a computer, with different performance:

Registers Small memory units built on the CPU (bytes, 1 cycle)

L1 Cache Last main memory access results (kB, 2-3 cycles)

L2 Cache (MB, 10 cycles)

Memory The usual RAM (GB, 100 cycles)

Storage Disks (100GB, TB, > 1Mcycles)

Takeaway: Use the registers as much as possible.

Memory Hierarchy

The stack and the heap are not a different physical memory level.

- They live inside RAM.
- logical structures used by programs.

Cache is invisible to the compiler (hardware managed)

No explicit instructions like: “put this variable in L1 cache”

Memory Hierarchy

The stack and the heap are not a different physical memory level.

- They live inside RAM.
- logical structures used by programs.

Cache is invisible to the compiler (hardware managed)

No explicit instructions like: “put this variable in L1 cache”

Takeaway: Use the registers as much as possible.

Register Overflow

What if there are not enough registers? Use the main memory, but how?

Recursion:

Without Each name is bound once. It can be statically allocated a single unit of main memory. (Cobol, Concurrent Pascal, Fortran (unless **recursive**)).

With A single name can be part of several concurrent bindings. **Memory allocation must be dynamic.**

Memory Allocation

Depending on the persistence, several models:

Global Global objects, whose liveness is equal to that of the program
(e.g., **static** variables in C)

Automatic Liveness is bound to that of the host function
(e.g., **auto** variables in C)

Heap liveness is independant from function liveness:

User Controlled

malloc/free (C), **new/dispose** (Pascal),
new/delete (C++) etc.

Garbage Collected

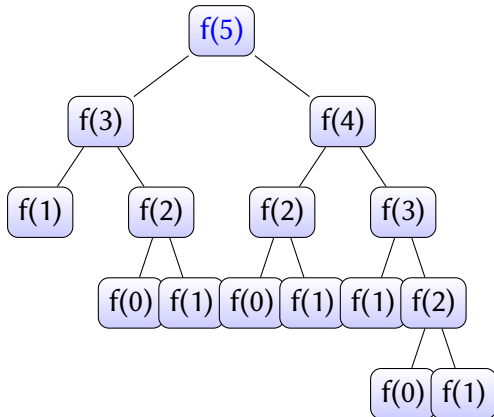
With or without **new**
(**lisp**, Smalltalk, ML, Haskell, Tiger, Perl etc.).

Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

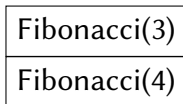
The complete history of calls: *is the depth first traversal of the call tree*

Fibonacci(5)

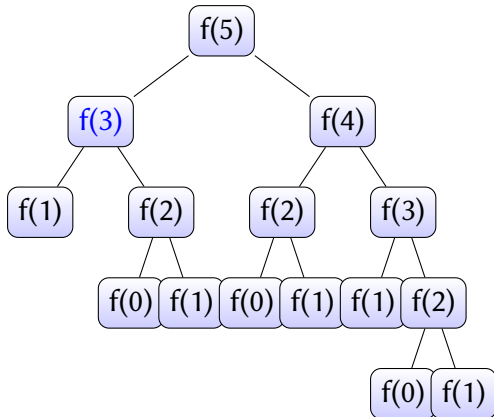


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.



The complete history of calls: *is the depth first traversal of the call tree*

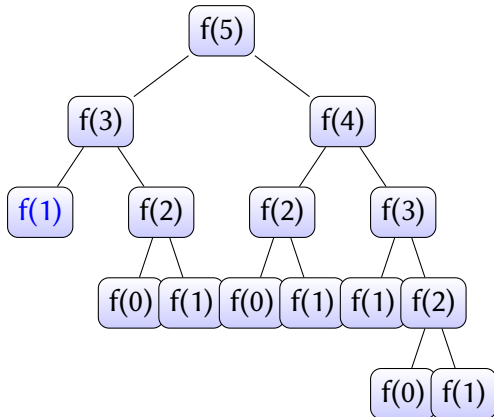


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

Fibonacci(1)
Fibonacci(2)
Fibonacci(4)

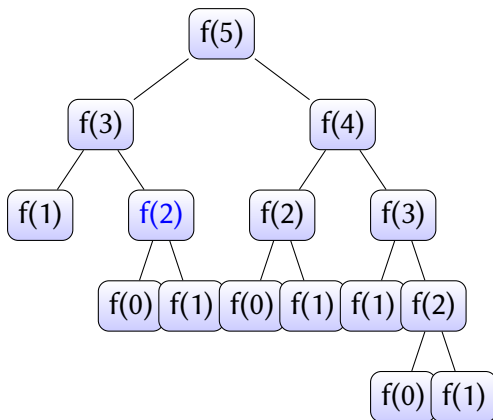
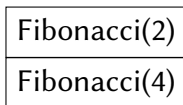
The complete history of calls: *is the depth first traversal of the call tree*



Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

The complete history of calls: *is the depth first traversal of the call tree*

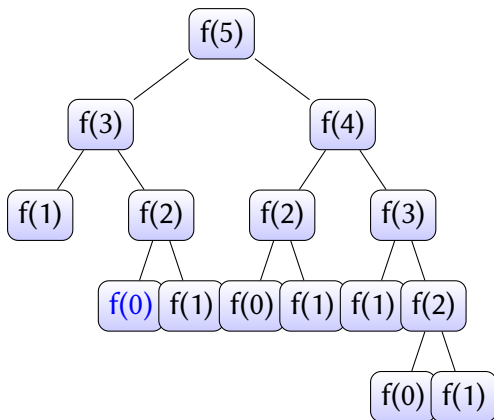


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

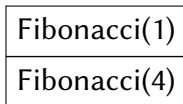
Fibonacci(0)
Fibonacci(1)
Fibonacci(4)

The complete history of calls: *is the depth first traversal of the call tree*

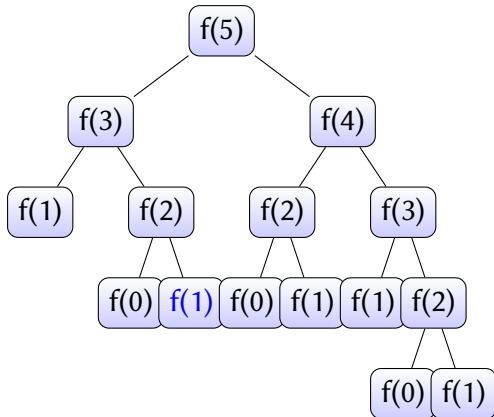


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.



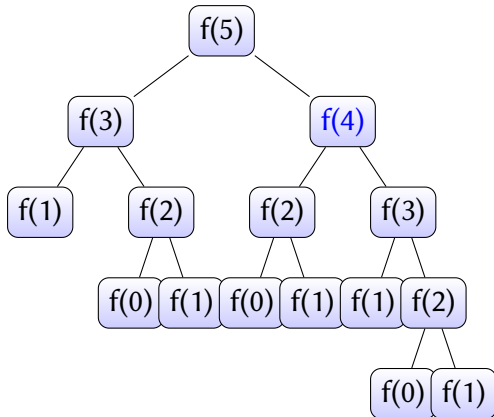
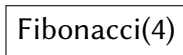
The complete history of calls: *is the depth first traversal of the call tree*



Stack Management

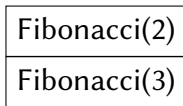
Function calls is a last-in first-out process: properly represented by a stack.

The complete history of calls: *is the depth first traversal of the call tree*

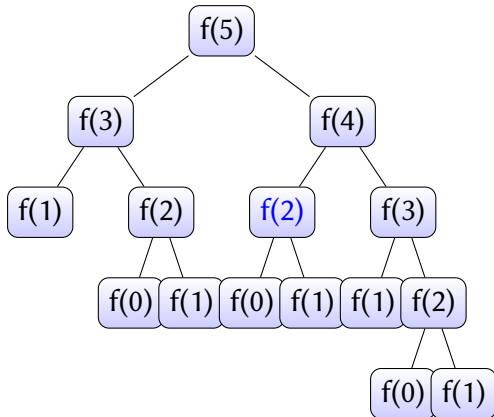


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.



The complete history of calls: *is the depth first traversal of the call tree*

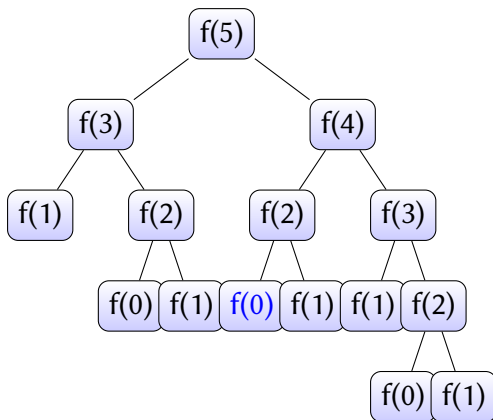


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

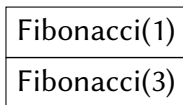
Fibonacci(0)
Fibonacci(1)
Fibonacci(3)

The complete history of calls: *is the depth first traversal of the call tree*

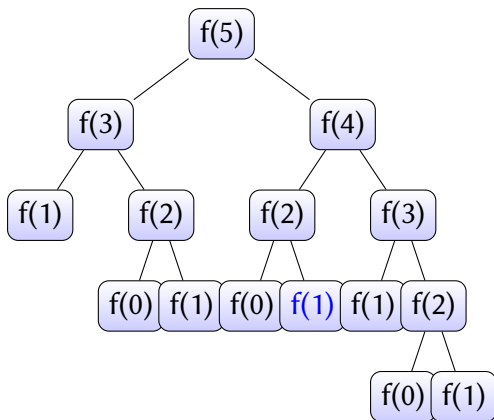


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.



The complete history of calls: *is the depth first traversal of the call tree*

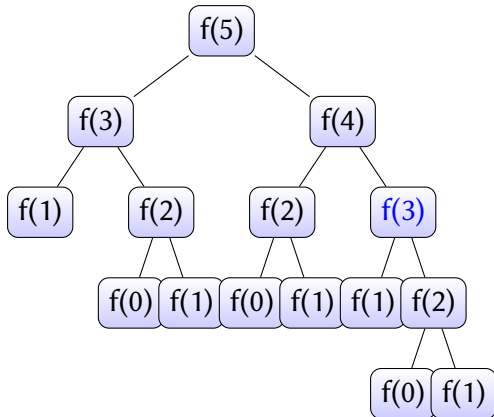


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

The complete history of calls: *is the depth first traversal of the call tree*

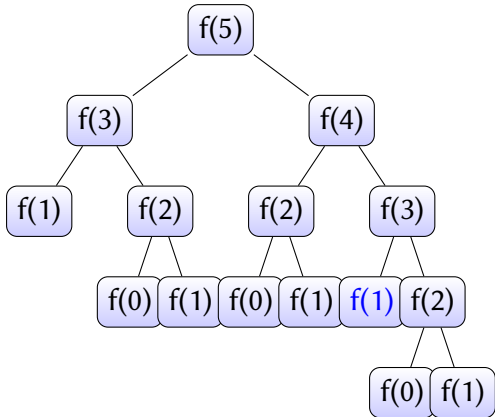
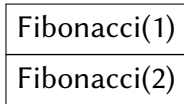
Fibonacci(3)



Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

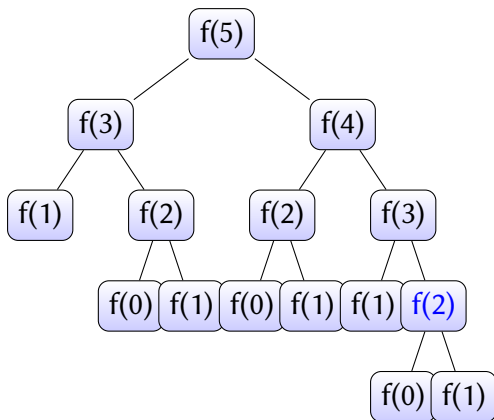
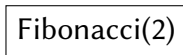
The complete history of calls: *is the depth first traversal of the call tree*



Stack Management

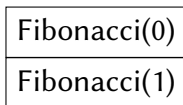
Function calls is a last-in first-out process: properly represented by a stack.

The complete history of calls: *is the depth first traversal of the call tree*

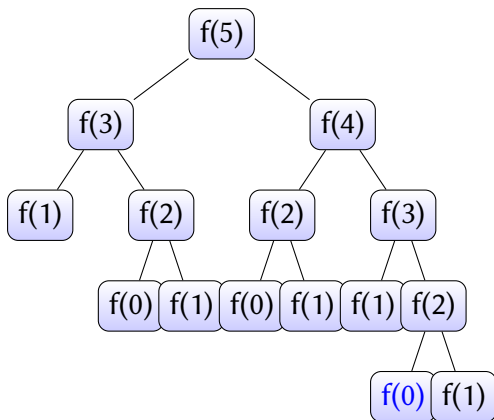


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.



The complete history of calls: *is the depth first traversal of the call tree*

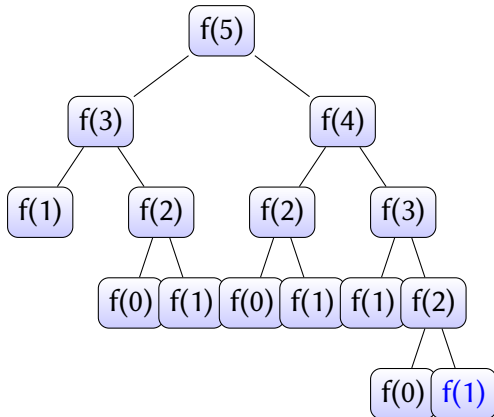


Stack Management

Function calls is a last-in first-out process: properly represented by a stack.

The complete history of calls: *is the depth first traversal of the call tree*

Fibonacci(1)



Activation Blocks

- In recursive languages, a single routine can be “opened” several times concurrently.
- An **activation** designates one single instance of execution.
- Automatic variables are bound to the liveness of the activation.
- Their location is naturally called **activation block**, or **stack frame**.

Activation Blocks - Example

- Each recursive call creates a **new activation block**
- Local variable **x** exists in **several instances**

```
function f(n) (  
  let x := n in           // local variable  
  if (n > 0) then f(n-1) + x else x);  
  
print(f(3));
```

Execution creates stack frames:

- $f(3) \rightarrow x = 3$
- $f(2) \rightarrow x = 2$
- $f(1) \rightarrow x = 1$
- $f(0) \rightarrow x = 0$

Exercise - 1

Translate to hir

```
function f1(x) (  
    function f2(y) ( /*local function*/  
        x+y  
    )  
)
```

Temporaries for arguments

In our IR, i_0 , i_1 , i_2 , ... refer the the first, second, third ... argument of the current routine (he enclosing one).

How to access it's parent one? or its parent's parent?

Activation Blocks Contents

Data to store on the stack:

arguments incoming

local variables user automatic variables

return address where to **return**

saved registers the caller's environment to restore

temp compiler automatic variables, spills

static link when needed

Why Return Registers Are Caller-Saved

Fundamental Reason

A callee cannot know what the caller will do with the returned value.

- A function does not know:
 - ▶ whether its return value will be used later
 - ▶ whether the caller will immediately call another function
 - ▶ whether the value must survive several subsequent calls

Why Return Registers Are Caller-Saved

Fundamental Reason

A callee cannot know what the caller will do with the returned value.

- A function does not know:
 - ▶ whether its return value will be used later
 - ▶ whether the caller will immediately call another function
 - ▶ whether the value must survive several subsequent calls
- Only the **caller** knows the **liveness** of the returned value.

Why Return Registers Are Caller-Saved

Fundamental Reason

A callee cannot know what the caller will do with the returned value.

- A function does not know:
 - ▶ whether its return value will be used later
 - ▶ whether the caller will immediately call another function
 - ▶ whether the value must survive several subsequent calls
- Only the **caller** knows the **liveness** of the returned value.
- Therefore:
 - ▶ the caller must decide whether to **move or save** the result
 - ▶ the callee cannot preserve the return register in a useful way

Why Return Registers Are Caller-Saved

Fundamental Reason

A callee cannot know what the caller will do with the returned value.

- A function does not know:
 - ▶ whether its return value will be used later
 - ▶ whether the caller will immediately call another function
 - ▶ whether the value must survive several subsequent calls
- Only the **caller** knows the **liveness** of the returned value.
- Therefore:
 - ▶ the caller must decide whether to **move or save** the result
 - ▶ the callee cannot preserve the return register in a useful way

Consequence

Return registers are naturally treated as **caller-saved (volatile)**.

Otherwise, function composition and optimizations such as tail calls would become significantly more complex or inefficient.

Activation Blocks Layout

The layout is suggested by the constructor.
Usually the layout is from earliest known, to latest.

The stack of activation blocks is implemented as an array with

frame pointer the inner frontier of the activation block

stack pointer the outer frontier

Usually the stack is represented growing towards the bottom.

Why Do We Need Calling Conventions?

The Problem

Functions use the same physical resources:

- they are compiled separately
- can use the same register **at the same time**
- all use the same stack

Calling convention = contract between routines

Registers vs Stack: Motivation

- Every function activation could theoretically work **only on stack space**
 - ▶ allocate locals in the activation block
 - ▶ access via offsets from the stack pointer
- But assembly **cannot perform many operations** directly on memory
- Registers are **faster** than memory and necessary for efficient computation
- **But are shared**: multiple functions/activation may need the same registers

Solution: Calling Convention + Save Rules

- **Caller-save**: caller saves registers that it wants preserved across a call
- **Callee-save**: callee saves registers it will use and restore before returning

Find the bug

```
f:
    li    s0, 42
    print_int(s0)
    ret
```

Find the bug

```
f:
    li    s0, 42
    print_int(s0)
    ret
```

```
li    s0, 0
jal   f
print_int(s0)    // expects 0
```

What happens?

The function `f` destroys the `s0` callee-saved register without restoring it.

Find the bug

```
li    t0, 42  
jal   f  
print_int(t0)
```

Find the bug

```
li    t0, 42  
jal   f  
print_int(t0)
```

Observed output: 0 //expected 42

Find the bug

```
li    t0, 42
jal   f
print_int(t0)
```

Observed output: 0 //expected 42

```
// function f
f:
    li    t0, 0
    ret
```

What happens?

The caller failed to save the t0 caller-saved register across a call.

Responsibilities

Caller Responsibilities

- pass arguments
- save caller-saved registers if needed
- perform the call

Callee Responsibilities

- preserve callee-saved registers
- create and destroy stack frame
- return result

RISC-V Register Classes

Class	Registers	Who saves?
Caller-saved	t0--t6, a0--a7	Caller
Callee-saved	s0--s11	Callee
Special	ra, sp	Special rules

Caller-saved

- volatile
- may be clobbered by calls
- good for short lifetimes

Callee-saved

- preserved across calls
- must be saved/restored by callee
- good for long lifetimes

Function Call Protocol

Caller (before call)

- place arguments in `a0--a7`
- save caller-saved registers
- `jal` to callee

Callee (on entry)

- allocate stack frame
- save used callee-saved registers
- save `ra` if needed

Stack Basics

Important

The RISC-V stack grows downward.

- `sp` always points to the top of the stack
- stack must remain 16-byte aligned
- each function owns its stack frame

Typical Stack Frame Layout

Frame Layout

	saved fs regs	
	saved s regs	
	saved ra	
	local spills	

ABI Requirement

Stack pointer must be 16-byte aligned at call boundaries.

Frame Size Computation

Formula

$$\text{frame size} = 8 \times (\#s + \#fs + \text{ra?})$$

ABI Alignment

Final size must be rounded up to a multiple of 16 bytes.

Example:

- 3 saved s-registers
- 2 saved fs-registers
- no need to save ra

$$8 \times 6 = 48 \rightarrow 48(\text{to preserve alignment})$$

Prologue

Typical RISC-V prologue

```
addi sp, sp, -frame_size
sd    s0, offset(sp)
sd    s1, offset(sp)
sd    ra, offset(sp)
```

Purpose:

- allocate frame
- preserve callee-saved registers
- preserve return address

Epilogue

Typical RISC-V epilogue

```
ld    s0, offset(sp)
ld    s1, offset(sp)
ld    ra, offset(sp)
addi  sp, sp, frame_size
ret
```

Invariant

After the epilogue, the caller must see the machine exactly as before the call.

Around Calls

Caller-side protocol

```
let x = ... in
let y = ... in
...
f(a,b);
...
print(x + y)
```

Suppose:

- the temporary x is mapped to the physical register t0
- the temporary y is mapped to the physical register t1

Around Calls

Caller-side protocol

```
# save caller-saved regs
sd    t0, offset(sp)
sd    t1, offset(sp)
# pass arguments
mv    a0, arg0
mv    a1, arg1
jal   ra, callee # actual call
# restore caller-saved regs
ld    t0, offset(sp)
ld    t1, offset(sp)
# result is in a0 / fa03
```

Optimization

Only values that are *live across the call* must be saved/restored by the caller.

After Register Allocation

Important point

Register allocation is *not the end*. Once temporaries are mapped to physical registers, the code must be **patched to respect the calling convention**.

Why?

- allocator ignores call boundaries
- physical registers have ABI roles
- we must insert:
 - ▶ prologue / epilogue
 - ▶ save/restore of callee-saved
 - ▶ caller-save around calls

What Conventions Fixes

Before patching

```
t3 <- ...  
jal f  
... uses t3 ...
```

After patching

```
sd    t3, offset(sp)  
jal   f  
ld    t3, offset(sp)
```

Responsibilities

- build stack frame
- preserve callee-saved registers
- protect live caller-saved across calls
- ensure ABI compliance

Complete Function Lifecycle

- 1 Caller prepares arguments
- 2 Caller executes `jal`
- 3 Callee runs prologue
- 4 Function body executes
- 5 Callee runs epilogue
- 6 Control returns to caller

Compiler Pipeline

Liveness → Coloring → Convention → Final(?) RISC-V

To go further

Live Across Calls

Key Observation

Some temporaries remain live across function calls.

Allocator strategy:

- temps live across calls → prefer callee-saved
- short-lived temps → prefer caller-saved

Why?

Caller-saved registers may be clobbered by the call.

Caller-Save Handling

Theoretical Rule

Caller must save any caller-saved register that is live across a call.

Our Design

Our allocator avoids this by placing long-lived values in callee-saved registers.

Consequence:

- no save/restore around most calls
- cheaper generated code
- allocator does the heavy work

Shrink-wrapping - 1

Idea

Goal

Delay the prologue and advance the epilogue so the frame is created *only where needed*.

Classic approach

```
entry:
  prologue      <-- always executed
  ...
  body
  ...
exit:
  epilogue
```


Shrink-wrapping - 2

entry:

```
...  
if (needs frame) {  
    prologue      <-- moved down  
    ...  
    epilogue      <-- moved up  
}
```

- avoids useless saves/restores
- reduces stack traffic
- especially useful for cold paths

Stack-Passed Arguments

When are arguments passed on the stack?

- When there are **more arguments than available argument registers**
- When arguments are **too large** to fit in registers (e.g. large structs)

RISC-V example

- First arguments: a0 – a7
- Remaining arguments: placed in the **caller stack frame**

Important

- The **caller allocates space** for stack-passed arguments
- The **callee reads them at fixed offsets** from the stack pointer or frame pointer

The end.

You compiler is finished!

The start.

You compiler is inefficient!

Optimizations

ILPML source

```
function f(n, a, b)(  
  let i := 0 in  
  let s := 0 in  
  while (i < n) do  
    s := s + (a * b);  
    i := i + 1  
  done;  
  s  
)
```

Naive Generated Code

```
# n in a0, a in a1 and b in a2
li    t0, 0          # i
li    t1, 0          # s
loop:
    bge t0, a0, end
    mul t2, a1, a2
    add t1, t1, t2
    addi t0, t0, 1
    j    loop
end:
    mv   a0, t1
    ret
```

Naive Generated Code

```
# n in a0, a in a1 and b in a2
li    t0, 0          # i
li    t1, 0          # s
loop:
    bge t0, a0, end
    mul t2, a1, a2
    add t1, t1, t2
    addi t0, t0, 1
    j    loop
end:
    mv   a0, t1
    ret
```

Problem

Expensive multiplication executed at every iteration.

After Loop Invariant Code Motion

```
li    t0, 0
li    t1, 0
mul   t2, a1, a2    # moved outside the loop
loop:
bge   t0, a0, end
add   t1, t1, t2
addi  t0, t0, 1
j     loop
end:
mv    a0, t1
ret
```

Effect

- removes redundant computation
- reduces loop body latency
- major speedup for large n

Real Motivation for LICM: ISA Constraints

Important

Even when the source program is well written, the generated code may contain loop-invariant work because of **instruction set constraints**.

Example (ILPML)

```
while (i < n) do
  s := s + (x + 15000);
  i := i + 1
end
```

Why can this be inefficient after instruction selection?

- RISC-V has limited immediate forms
- there is no general `addi` / `mul` for arbitrary constants
- large constants must be **loaded into temporaries**

Naive selected code

```
loop:
    li    t2, 15000          # loop invariant load
    add   t3, x, t2
    add   s, s, t3
    ...
```

Why can this be inefficient after instruction selection?

- RISC-V has limited immediate forms
- there is no general `addi` / `mul` for arbitrary constants
- large constants must be **loaded into temporaries**

Naive selected code

```
loop:
    li    t2, 15000          # loop invariant load
    add   t3, x, t2
    add   s, s, t3
    ...
```

LICM Opportunity

The load of constant 15000 is loop-invariant and can be moved **before the loop**.

Must-Have Optimization: Dead Code Elimination

ILPML

```
let x := a + b in  
let y := x * 4 in  
let z := y + 1 in  
x
```

Optimization

Dead Code Elimination

- y and z are never used (frequent in *full-expression* languages)
- remove useless computations and temporaries

Must-Have Optimization: Constant Propagation

ILPML

```
let x := 4 in  
let y := x + 3 in  
y * 2
```

Naive generated code

```
li    t0, 4  
addi  t1, t0, 3  
slli  a0, t1, 1  
ret
```

Optimization

Constant Propagation + Folding compute constants at compile time

result becomes:

```
li a0, 14  
ret
```

Must-Have Optimization: Array Address Optimization

ILPML

```
let i := 0 in
while (i < n) do
  s := s + A[i];
  i := i + 1
end
```

Naive generated pattern

```
loop:
slli t1, i, 3           # compute i * element_size
add  t2, base, t1
ld   t3, 0(t2)
add  s, s, t3
addi i, i, 1
```

Must-Have Optimization: Array Address Optimization

Optimization

Strength Reduction Optimization

- maintain running pointer instead of recomputing address
- replace multiply by simple increment

```
mv    t2, base           # running pointer = &A[0]
loop:
ld     t3, 0(t2)          # load current element
add    s, s, t3
addi   t2, t2, 8          # advance pointer (element_size)
addi   i, i, 1
blt    i, n, loop
```

Must-Have Optimization: Function Inlining

ILPML

```
function inc(x) (x + 1)
```

```
let y := inc(a) in  
inc(y)
```

Optimization

Inlining

- removes call / return overhead
- exposes new optimization opportunities
- enables constant propagation and register reuse

Must-Have Optimization: Tail Call Optimization

ILPML

```
function f(x) (  
  g(x)  
)
```

Naive generated code

```
mv    a0, x  
jal   g  
ret
```

Must-Have Optimization: Tail Call Optimization

Optimization

Tail Call Optimization

- replace call + return by direct jump
- avoids growing the stack
- essential for compiling functional / recursive programs efficiently

Optimized pattern:

```
mv a0, x  
j g
```

Tail Call Situation

ILPML (tail recursion)

```
function sum(n, acc)(  
  if n = 0 then acc  
  else sum(n-1, acc+n)  
)
```

Naive generated pattern

```
# prepare recursive call  
addi a0, n, -1  
add a1, acc, n  
jal sum  
ret
```

Each recursive call creates a new stack frame.

Tail Call Optimization

Observation

The recursive call is the **last action** of the function.

Optimization idea

- no work remains after the call
- current activation block is useless
- we can **reuse the same stack frame**

Optimized pattern

```
addi a0, n, -1
add  a1, acc, n
j    sum
```

Stack depth becomes constant.